

Plaid Vendor Security Questionnaire — Responses

Submitted by: Mark Stolte, Founder — markstolte02@gmail.com

Date: 2026-07-05

Basis for answers: current codebase ([Mstolte02/penny-pilot](#)), the policy set in [docs/security/](#), and direct confirmation from the founder on items not visible in code (admin-account MFA status, device security practices).

Answers are written to be accurate about a pre-launch, single-founder company, including saying "not yet" where that's the honest answer. See [docs/security/README.md](#) for organizational context.

1. Documented information security policy, operationalized to identify/mitigate/monitor risk

Yes. See [information-security-policy.md](#) and the full policy set it indexes in [README.md](#):

- [Access Control Policy](#)
- [Password & MFA Policy](#)
- [Data Classification & Handling](#)
- [Encryption Standards](#)
- [Secure Software Development Lifecycle](#)
- [Vulnerability Management](#)
- [Incident Response Plan](#)
- [Business Continuity & Disaster Recovery](#)
- [Logging & Monitoring](#)
- [Vendor Risk Management](#)
- [Employee Offboarding](#)
- [Risk Assessment Process](#)

These were authored and first reviewed 2026-07-05, sized to a one-person team, and state plainly which controls are already operating versus planned with a target date — rather than describing an enterprise program that doesn't exist yet. The [Risk Assessment Process](#) is the mechanism by which risks are identified and tracked going forward (it includes a live risk register with target dates).

2. Access controls in place to limit access to production assets and sensitive data

(select all that apply — answered per item, since several are partial)

Control	Status
---------	--------

A defined and documented access control policy	Yes — access-control-policy.md .
RBAC	No. Team of one; there's no multi-role staff permission model yet. End-user data isolation is enforced differently — via Postgres Row Level Security scoped to <code>auth.uid()</code> on every user-owned table — which prevents cross-user access but isn't role-based access control for staff. RBAC among staff is planned for when a second person joins.
Periodic access reviews and audits	Yes, self-conducted. Access Control Policy defines a review cadence (on any role/vendor change, and at minimum annually). This is founder self-review today, not an independent third-party audit.
Automated de-provisioning/modification of access for terminated/transferred employees	No. Employee Offboarding defines a manual checklist, since there are no employees/contractors yet to de-provision. Automation is not built because there's nothing to automate against today; it will be evaluated once there's a second person with access.
Zero trust access architecture	Not formally. Every backend request (Edge Functions, Postgres via RLS) is authenticated per-request using a Supabase-issued JWT regardless of network origin — no implicit trust based on network location — but this hasn't been implemented or documented as a named Zero Trust program.
Centralized identity and access management solution	Partial. For end users: yes — Supabase Auth is the single, centralized identity provider (via Apple/Google OAuth). For internal/admin access to infrastructure (GitHub, Supabase dashboard, Plaid dashboard, Google Cloud/Play Console), no — each is a separate vendor login today, which is a gap for when the team grows past one person.
Use of OAuth tokens or TLS certificates for non-human authentication	Yes. Every Supabase Edge Function requires a valid Supabase-issued JWT on the Authorization header (<code>supabase/functions/_shared/supabase.ts</code>); all calls to Plaid's API happen server-side over TLS. No service-to-service call in this system relies on a shared static secret alone without TLS.

3. MFA for consumers on mobile/web before Plaid Link is surfaced

Authentication is required and backend-enforced before Plaid Link can open; a separate, Penny-Pilot-specific MFA challenge is not layered on top of that.

Concretely: `plaid-create-link-token` (the function that must succeed before the Plaid Link SDK can be opened on-device) requires a valid Supabase session and throws "Sign in before connecting a bank" otherwise (`supabase/functions/_shared/supabase.ts`, `supabase/functions/plaid-create-link-token/index.ts`). Sign-in itself is exclusively via

Apple or Google OAuth — Penny Pilot has no password of its own to protect. Any MFA a user has enabled on their Apple ID or Google Account applies transitively to this sign-in, since Penny Pilot never sees or bypasses that provider's own authentication flow. Penny Pilot does not currently add an additional, separate MFA prompt of its own before opening Plaid Link.

4. MFA for access to critical systems that store/process consumer financial data

Not fully confirmed as of this review. Policy requires MFA on every admin system that can reach consumer financial data — GitHub, the Supabase dashboard, the Plaid dashboard, and Google Cloud/Play Console (see [Password & MFA Policy](#)) — but as the sole operator, the founder has not verified enrollment is actually turned on across all of them. This is the single highest-priority action item coming out of this review: enabling MFA on all four is a same-day fix with no dependencies, and is being done as a direct follow-up to this questionnaire rather than left as a stated policy with unverified practice.

5. TLS 1.2+ encryption in transit between clients and servers

Yes. All communication with Supabase (Postgres via PostgREST, Auth, Edge Functions) and with Plaid's API happens over HTTPS/TLS via those vendors' SDKs/endpoints; the codebase contains no cleartext HTTP backend calls. Plaid Link's OAuth redirect also uses an HTTPS Universal Link rather than a custom URL scheme, per Plaid's own OAuth requirement. See [Encryption Standards](#).

6. Encryption at rest for consumer data received from the Plaid API

Partially — the token is field-level encrypted; the broader synced data relies on platform-level encryption, not application-layer field encryption. Specifically:

- The Plaid **access token** itself — the single most sensitive value — is encrypted with **AES-256-GCM** using an application-managed key (`PLAID_TOKEN_ENCRYPTION_KEY`) that exists only as a Supabase Edge Function secret, before being written to `plaid_items.access_token_ciphertext` (`supabase/functions/_shared/plaid.ts`). It is never stored or transmitted in plaintext outside the exchange step.
- The **transaction and account data** Plaid returns (merchant, amount, balances, institution/account metadata) is stored in Supabase Postgres and protected by (a) Supabase's platform-level storage encryption at rest, per Supabase's own documented infrastructure, and (b) Row Level Security limiting reads to the owning user — but it does not have a separate application-layer field encryption on top of that, the way the access token does. See [Data Classification & Handling](#) and [Encryption Standards](#) for the full breakdown by field.

7. Vulnerability scanning of employee/contractor machines and production assets

Not yet on either surface, with one important caveat: there are no self-managed production servers to scan. All production infrastructure (Postgres, Auth, Edge Functions) runs on Supabase's managed platform — there is no Penny-Pilot-operated VM,

container host, or bare-metal instance to run a vulnerability scanner against; scanning the underlying platform is Supabase's responsibility as the infrastructure provider. On the founder's development machine, no dedicated endpoint-protection or vulnerability-scanning tool is confirmed in place today — only default OS protections are assumed, unverified.

Practices actually in place today (select-all-that-apply, answered honestly):

- ☒ Dependency / software-composition-analysis scanning — GitHub Dependabot security alerts (enabled 2026-07-05 as part of this review) plus ad hoc `npm audit`.
- ☐ Endpoint protection / vulnerability scanning tool on developer machines — not confirmed; action item to enable full-disk encryption and confirm automatic OS security updates on the development machine as an immediate, low-cost fix.
- ☐ Network/infrastructure vulnerability scanning of production assets — not applicable today (no self-managed servers); inherited from Supabase's platform security.
- ☐ Scheduled third-party penetration test — not yet performed; planned before production users are onboarded at scale.

8. Privacy policy for the application where Plaid Link will be deployed

Not yet published. This is a real, disclosed gap rather than an oversight papered over — the app does not currently have a published privacy policy. This needs to exist before Plaid Link is used with real (non-Sandbox) consumer accounts, and drafting one (covering what's collected, Plaid's role, retention, and user rights) is a direct, near-term follow-up to this questionnaire.

9. Consent for collection, processing, and storage of consumer data

Not yet a dedicated Penny Pilot consent step. Plaid Link itself presents Plaid's own end-user disclosure/consent panel before linking an account (standard Plaid Link behavior, outside Penny Pilot's control), and Apple/Google present their own OAuth consent screens for the identity data they share — but Penny Pilot does not yet capture its own explicit consent (e.g., an "I agree to the Privacy Policy and Terms" step) before account creation or data collection. This should ship alongside the privacy policy in item 8.

10. Data deletion/retention policy compliant with applicable privacy laws, reviewed periodically

Partially defined, not yet fully implemented or independently mapped to specific laws.

[Data Classification & Handling](#) documents that deleting a user's `profiles` row cascades to every dependent table via `on delete cascade` foreign keys in the schema (`supabase/migrations/0001_initial_schema.sql`). Three concrete gaps remain, disclosed rather than hidden:

1. Account deletion does not yet trigger a Plaid `/item/remove` call, so the underlying bank connection at Plaid isn't automatically revoked when a user deletes their account — only the local database rows are removed today.
2. There is no standalone retention-schedule document mapping specific data categories to specific legal retention requirements (e.g., CCPA, GLBA, state-level requirements) —

today's retention behavior is "as long as the account exists," not a law-specific schedule.

3. There is no self-service, in-app data export or deletion request flow yet for a user who wants their data before or instead of deleting their account outright.

Target: close all three before Plaid Link is used with real consumer accounts in production, and review this section on the same cadence as the rest of the [Risk Assessment Process](#) once implemented.

Follow-up actions opened by this questionnaire

Tracked in [Risk Assessment Process](#)'s risk register; the two most urgent, both same-day fixes:

1. **Enable MFA now** on GitHub, the Supabase dashboard, the Plaid dashboard, and Google Cloud/Play Console — currently unverified, addressed directly by question 4 above.
2. **Enable full-disk encryption (FileVault) and confirm automatic OS security updates** on the development machine — addresses question 7's device-security gap at effectively no cost.

Larger, pre-production-launch items: publish a privacy policy, add an in-app consent step, and implement Plaid item revocation + a real retention schedule on account deletion (questions 8–10).